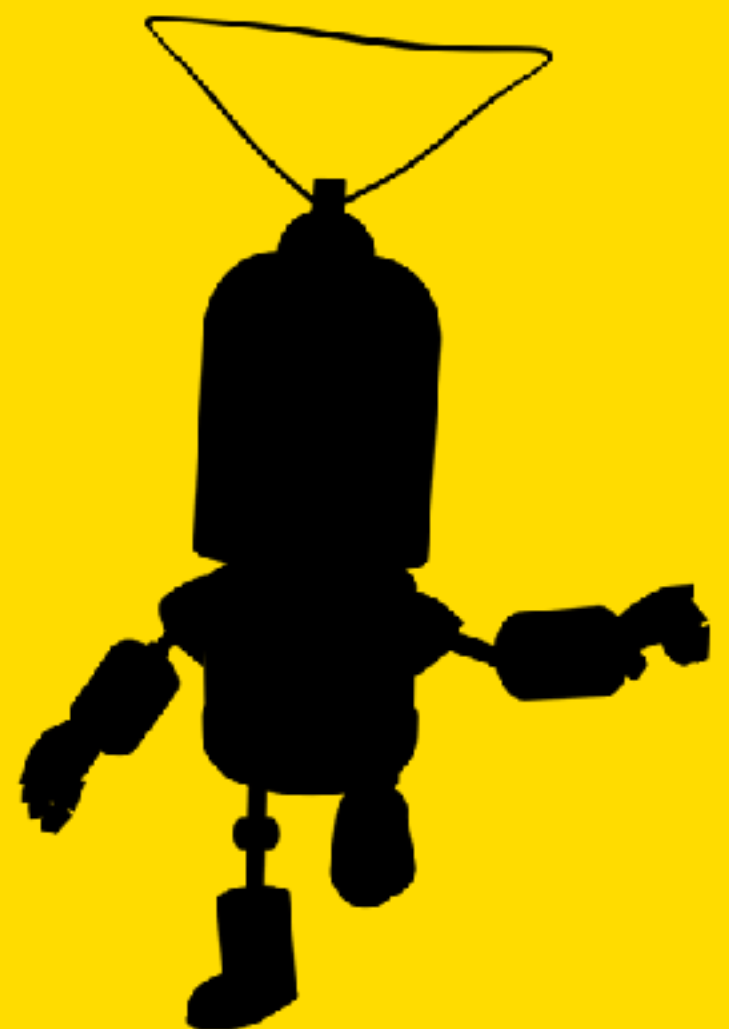


Knowledge representation

COMP9414 Artificial Intelligence

Oliver Obst



Overview



1	5/6	Introduction. Agents & search		
2	12/6	Planning		release assignment #1
3	19/6	Knowledge representation		
4	26/6	Neural networks		
5	3/7	Reinforcement learning		deadline assignment #1 (6 Jul)
6	10/7			release assignment #2
7	17/7	Reasoning with uncertainty		
8	24/7	Computer vision		
9	31/7	Foundation models		deadline assignment #2 (31 Jul)
10	7/8	Human-aligned intelligent robotics		

After this lecture

You should be able to:

- explain how logical facts extend the state descriptions used in planning;
- keep syntax, semantics, and proof/search separate;
- evaluate small propositional formulas in a truth assignment;
- describe entailment as a relationship between possible worlds;
- distinguish semantic entailment from derivation inside a proof system;
- convert small formulas into clauses;
- describe the Davis–Putnam–Logemann–Loveland (DPLL) algorithm as backtracking search over partial truth assignments;
- use resolution to prove propositional entailment;
- recognise how first-order logic writes object-level knowledge more compactly.

From planning to logic

Slides material and acknowledgement:

- ALMA Chapter 7 logical agents slides for knowledge bases, entailment, CNF, Horn clauses, and propositional resolution.
- ALMA Chapter 8 and Stanford first-order logic material for the short FOL preview.
- Berkeley CS188 Lecture 8 on propositional logic and inference
- Stanford CS221 logic materials for CNF conversion, resolution, and the transition from propositional logic to first-order logic
- University of Basel and TUM propositional-resolution handouts for the refutation view, unsatisfiability, and the empty clause.
- The Python ai9414 DPLL and resolution demos for live examples and visual proof traces.

From planning to knowledge representation

Planning assumed that states could be represented.

Knowledge representation asks what those state descriptions mean.

- Search: which path reaches a goal?
- Planning: which actions transform one state into another?
- Logic: which facts and rules make a conclusion unavoidable?

STRIPS-style reminder

State= {AtRobot(Office), HasParcel,
LabLocked, HasKeycard}

Action

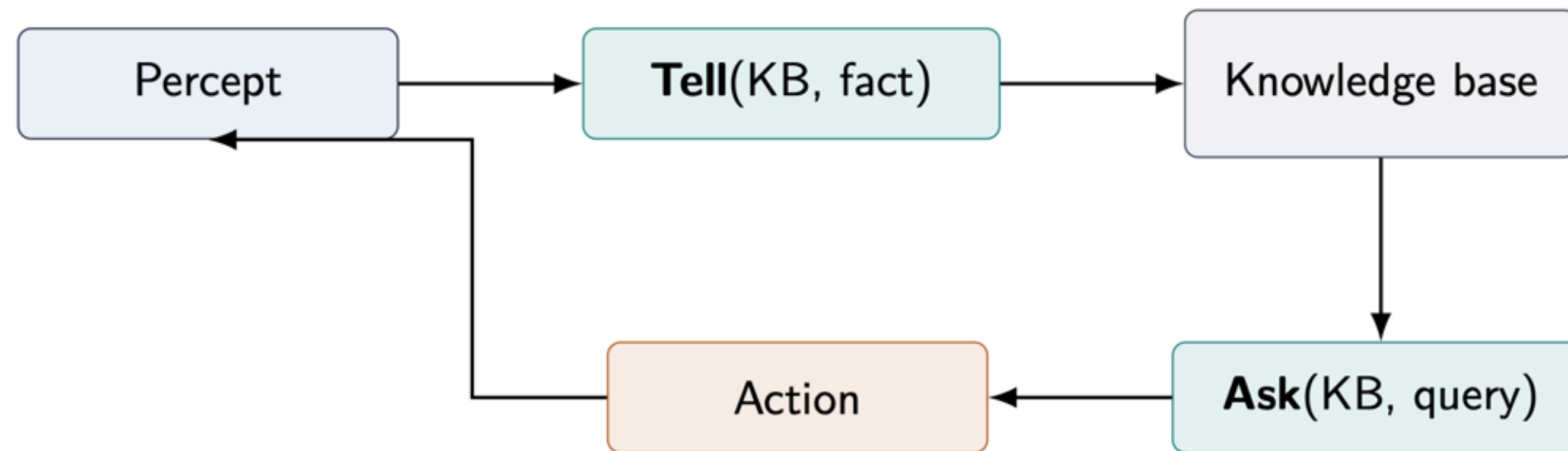
EnterLab

Preconditions:

AtRobot(LabDoor), HasKeycard

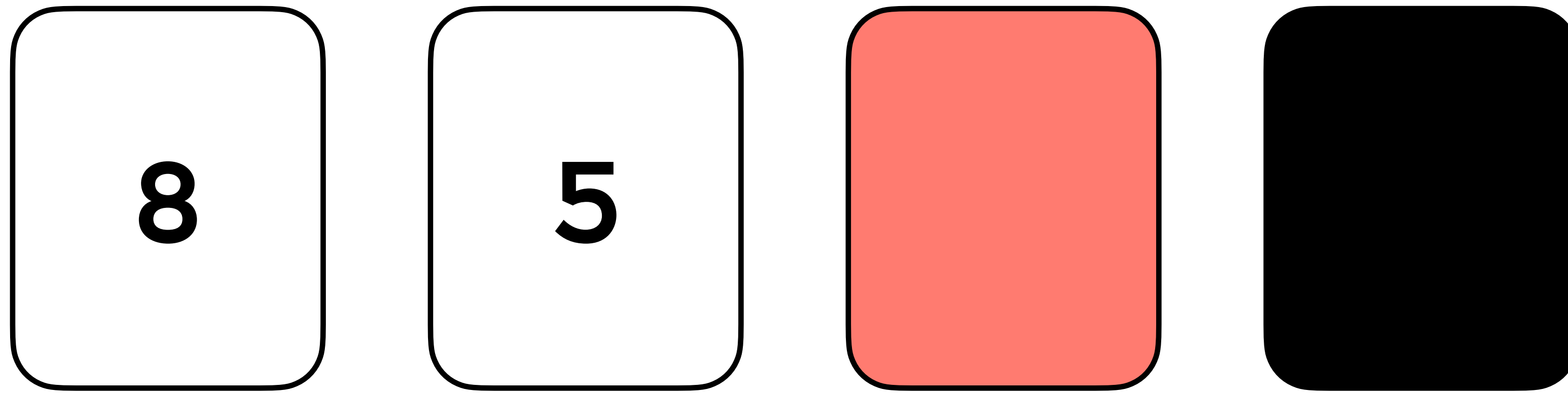
Effects: InLab

Knowledge-based agents pattern



- Tell: add facts, observations, or action records.
- Ask: query what follows from the knowledge base.
- The same inference method can be reused with different domain knowledge.

Warm-up logic test: 4 cards



Each card has a number on one side and a colour on the other.

Rule: If a card has an even number on one side, then the other side is red.

Which cards must you turn over to test whether the rule has been broken?

Proof patterns

Let $P =$ “the card has an even number”, $Q =$ “the card is red”

The rule is: $P \rightarrow Q$

Turn over 8:

$$\frac{P, P \rightarrow Q}{Q}$$

This is **modus ponens**:

If P implies Q , and P is true,
then Q must be true.

Turn over black:

$$\frac{\neg Q, P \rightarrow Q}{\neg P}$$

This is **modus tollens**:

If Q is false, and P implies Q ,
then P cannot be true.

Tempting mistakes

The other cards feel relevant, but the rule does not make them relevant.

Red card:

$$\frac{Q, P \rightarrow Q}{P}$$

This is not valid. This mistake is called **affirming the consequent**:

The rule says even cards must be red; it does not say only even cards are red.

5 card:

$$\frac{\neg P, P \rightarrow Q}{\neg Q}$$

This is not valid either. This mistake is called **denying the antecedent**:

The rule says nothing about odd cards.

Back to the office (robot)

Story:

- A robot needs to deliver a parcel to a lab.
- A keycard lets the robot open the lab door.
- Opening the door lets the robot enter the lab.
- If the robot has the parcel and can enter the lab, delivery is possible.

Propositional symbols

K : robot has a keycard

O : robot can open the lab door

E : robot can enter the lab

P : robot has the parcel

D : parcel is delivered

The kind of questions logic can answer

Knowledge base

$$K \rightarrow O$$

$$O \rightarrow E$$

$$P \wedge E \rightarrow D$$

K

P

Question

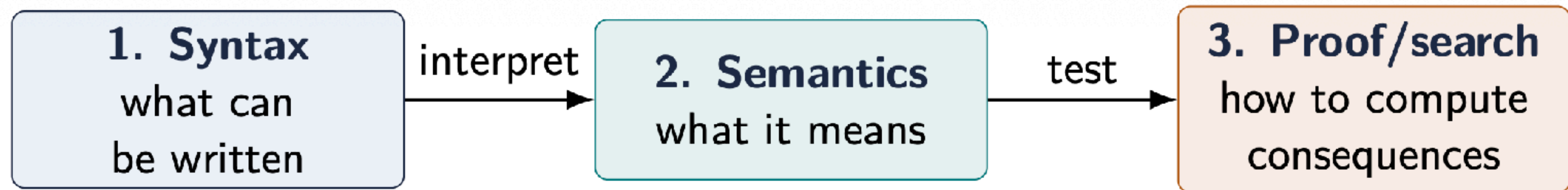
$$KB \models D ?$$

If the robot has the keycard and the parcel, must the parcel be deliverable?

Logic gives a precise meaning to “must”.

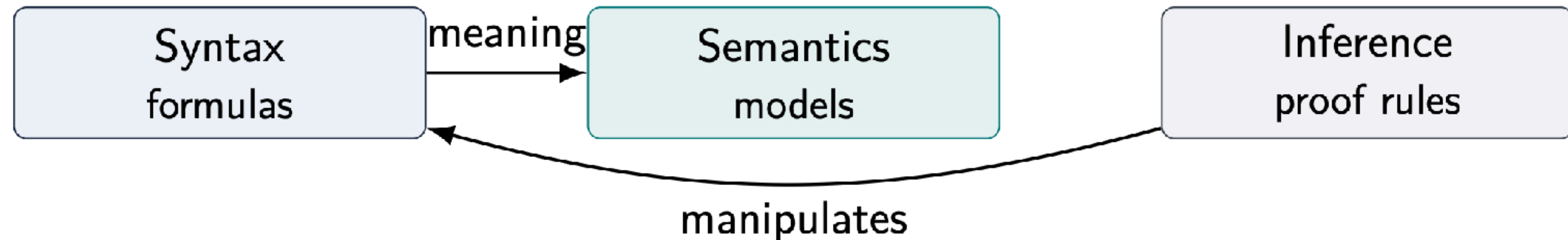
$KB \models D$ asks: must D be true in every world allowed by the KB?

Layers of “logic”



We move from writing sentences, to giving them meaning in possible worlds, to using algorithms that recover the consequences.

Syntax, semantics, and inference



- Syntax says which strings are legal formulas.
- Semantics says when formulas are true in a possible world.
- Inference rules derive new formulas from old formulas.
- Keep the layers separate; connect them with soundness and completeness.

Same “sentence”, 3 questions

$$K \wedge P \rightarrow D$$

Syntax

Is this a legal propositional formula?

Yes: it is built from symbols and connectives.

Semantics

In which possible worlds is it true?

It is false only when K and P are true but D is false.

Proof/search

Can an algorithm derive or refute something from it?

That depends on the chosen inference method.

Symbols and propositional syntax

A formula is a legal expression built from propositional symbols and connectives.

Atomic propositions:

K, O, E, P, D, A, B

Each symbol is either true or false in a model.

Connectives

Symbol	Name	Read as
$\neg f$	negation	not f
$f \wedge g$	conjunction	f and g
$f \vee g$	disjunction	f or g
$f \rightarrow g$	implication	if f , then g
$f \leftrightarrow g$	biconditional	f iff g

The disjunction: $\vee$ is inclusive-or

Truth conditions

f	g	$\neg f$	$f \wedge g$	$f \vee g$	$f \rightarrow g$	$f \leftrightarrow g$
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

In propositional logic, $A \rightarrow B$ is false only when A is true and B is false. It does not say that A causes B . It only rules out worlds where A holds but B does not.

Parsing a formula

$$((\neg K \vee O) \rightarrow E) \wedge (P \rightarrow D)$$

Built recursively

K, O, E, P, D are atomic formulas.

$\neg K$ and $(\neg K \vee O)$ are formulas.

$(\neg K \vee O) \rightarrow E$ and $P \rightarrow D$ are formulas.

The whole sentence is a conjunction.

Truth needs context

- The formula's shape tells us how it is built, not whether it is true.
- To evaluate it directly, we need a model: a complete truth assignment.
- To prove it, we need premises in a KB plus inference rules.

Evaluate the formula

Model:

$$\{K : T, O : F, E : T, P : T, D : T\}$$

Sentence:

$$((\neg K \vee O) \rightarrow E) \wedge (P \rightarrow D)$$

What is the truth value?

- A. True
- B. False
- C. Not enough information to determine
- D. It depends on other symbols in the knowledge base

Answer: true

$$\begin{aligned} ((\neg T \vee F) \rightarrow T) \wedge (T \rightarrow T) &= ((F \vee F) \rightarrow T) \wedge (T \rightarrow T) \\ &= (F \rightarrow T) \wedge (T \rightarrow T) \\ &= T \wedge T \\ &= T \end{aligned}$$

Both implications have T on the right-hand side. In propositional logic, $f \rightarrow T$ is always true, so the sentence becomes $T \wedge T$.

Models

A **model** in propositional logic is a complete assignment of truth values to all relevant symbols.

$$w = \{K : T, O : T, E : T, P : T, D : F\}$$

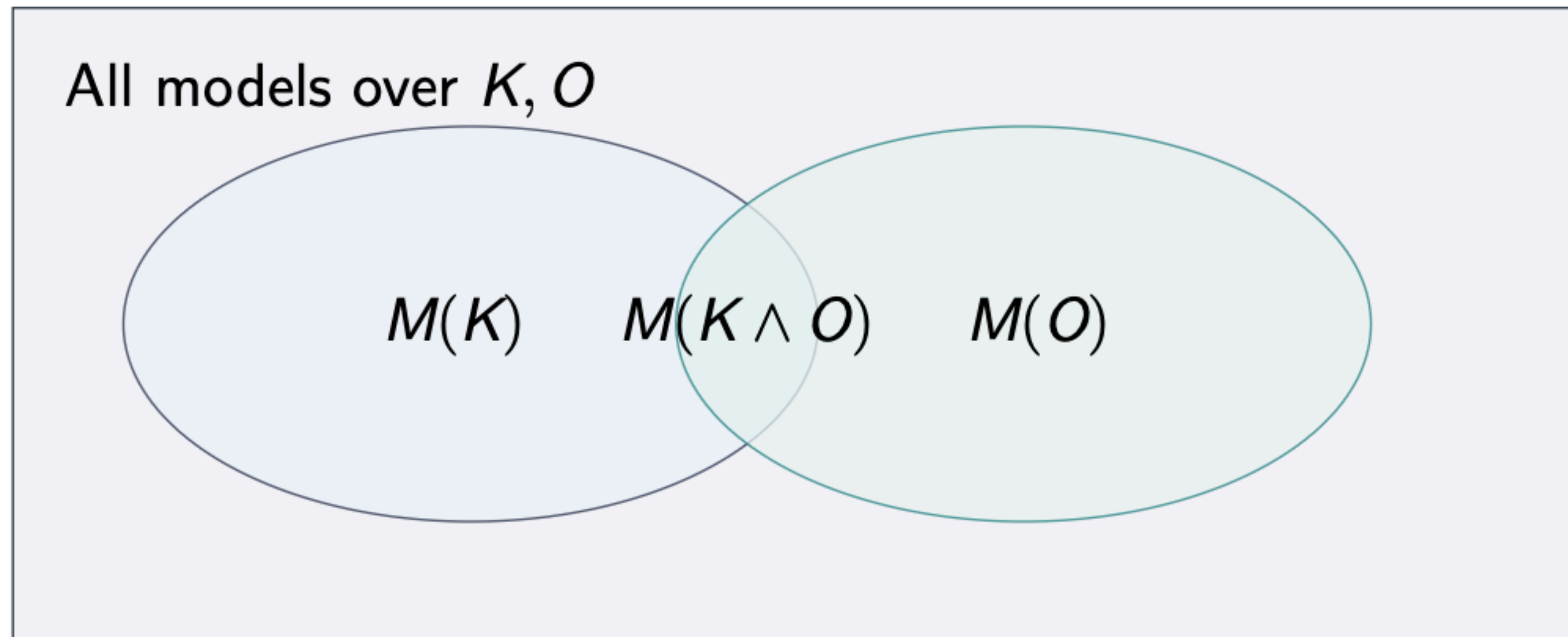
A planning state is one possible description of the world.

A logical model is a complete truth assignment.

A knowledge base usually describes a set of possible worlds, not just one world.

n symbols $\Rightarrow 2^n$ possible models

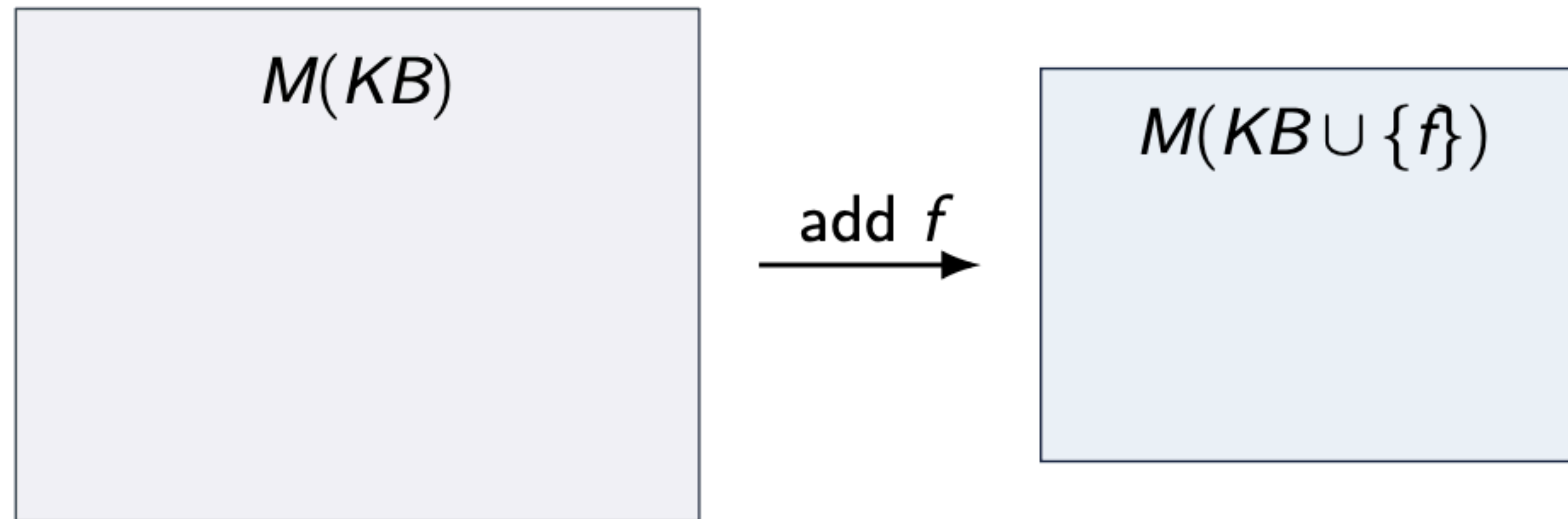
A formula picks out a set of models



$M(f)$ is the set of models where formula f is true.

$M(KB)$ is the set of models satisfying every sentence in the knowledge base.

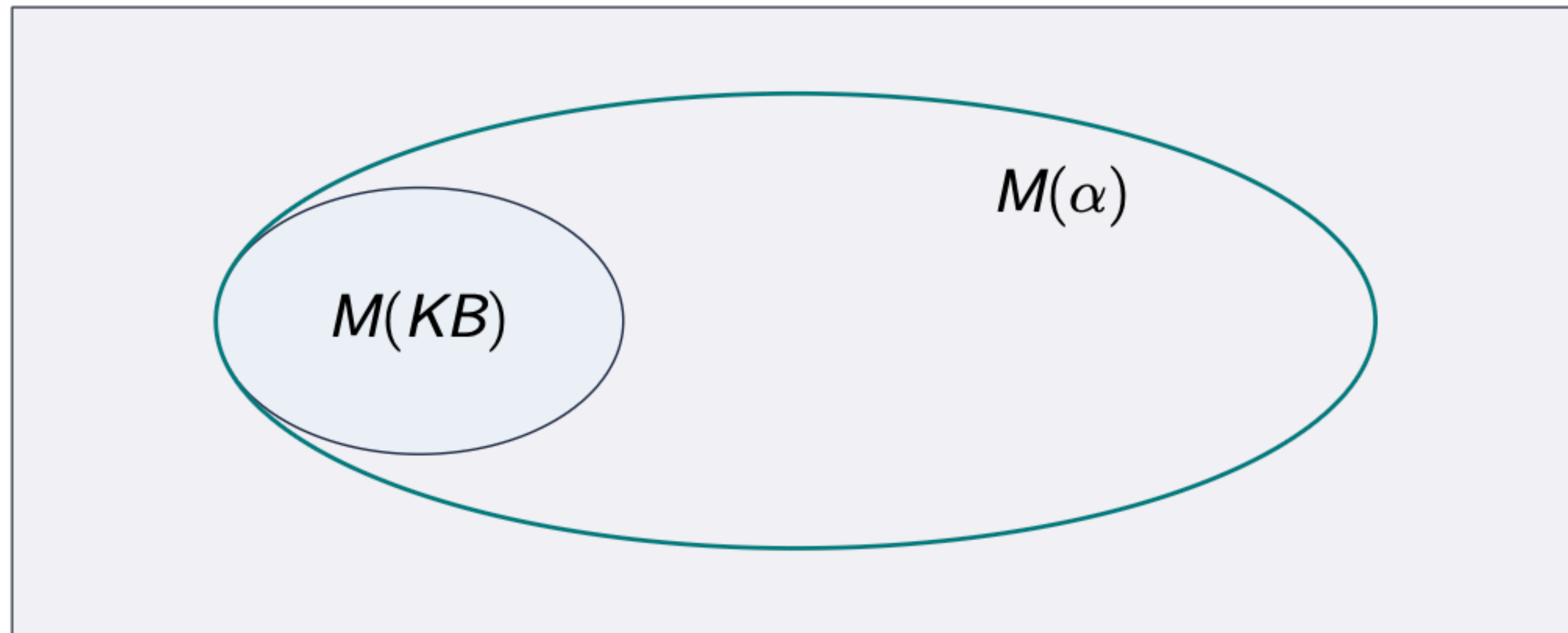
Adding knowledge shrinks possibility



$$M(KB \cup \{f\}) = M(KB) \cap M(f)$$

Adding a fact can rule out worlds. It can never add new possible worlds.

Entailment



$$KB \models \alpha \quad \text{iff} \quad M(KB) \subseteq M(\alpha)$$

KB is a knowledge base: a set of sentences. α is a query sentence.

Entailment means every model of KB also satisfies α .

Model checking entailment

Let $KB = \{K, K \rightarrow O, O \rightarrow E\}$. Query: $KB \models E$?

Model checking enumerates all truth assignments:

K	O	E	KB fragment true?	Query E true?
F	F	F	no	F
F	F	T	no	T
F	T	F	no	F
F	T	T	no	T
T	F	F	no	F
T	F	T	no	T
T	T	F	no	F
T	T	T	yes	T

It asks whether E is true in every model where KB is true.

This is sound and complete, but exponential in the number of symbols.

More model checking entailment

Let $KB = \{K, K \rightarrow O, O \rightarrow E\}$.

Which queries are entailed by KB?

- A. E
- B. $K \vee P$
- C. P
- D. Both E and $K \vee P$

Correct answer: D

- E is entailed by the chain $K \rightarrow O \rightarrow E$.
- $K \vee P$ is entailed because K is already true in every KB model.
- P is not entailed. The KB says nothing about whether the robot has the parcel.

The Wumpus world

The Wumpus world is a classic AI grid-world used to study reasoning under partial information.

World

- An agent explores a cave one square at a time.
- Some squares may contain pits.
- One square may contain the Wumpus.
- The agent wants to infer which nearby squares are safe.

Percepts

- A breeze means a pit is in an adjacent square.
- A stench means the Wumpus is in an adjacent square.
- No breeze is evidence that adjacent squares have no pits.

We use only the breeze–pit rule here, because it is enough to demonstrate entailment and resolution.

Case study: Wumpus-style reasoning

A tiny cave fragment

?	[2,2]	[3,2]
Ⓐ	?	[3,1]

Ⓐ marks the agent's current square. The question marks are hidden adjacent squares.

Symbols

$B_{x,y}$: breeze in square $[x, y]$

$P_{x,y}$: pit in square $[x,y]$

A percept is a fact. A cave rule is a sentence relating facts.

A Wumpus rule as logic

In the Wumpus world, a square is breezy exactly when at least one adjacent square contains a pit.

$$B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$$

If there is a breeze

If there is a pit

$$B_{1,1} \rightarrow (P_{1,2} \vee P_{2,1})$$

$$(P_{1,2} \vee P_{2,1}) \rightarrow B_{1,1}$$

The biconditional says both directions hold.

Observations shrink the model set

Suppose the agent starts at [1, 1] and perceives no breeze.

$$\neg B_{1,1}$$

Together with:

$$B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$$

this rules out every model where $P_{1,2}$ or $P_{2,1}$ is true.

$$\text{KB} \models \neg P_{1,2} \qquad \text{KB} \models \neg P_{2,1}$$

The same case as possible worlds

$P_{1,2}$	$P_{2,1}$	$B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$	$\neg B_{1,1}$ possible?
F	F	$B_{1,1}$ must be F	yes
F	T	$B_{1,1}$ must be T	no
T	F	$B_{1,1}$ must be T	no
T	T	$B_{1,1}$ must be T	no

Only one row survives the rule plus the observation.

$$\neg P_{1,2} \wedge \neg P_{2,1}$$

follows semantically.

Wumpus entailment

Let

$$\text{KB} = \{B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1}), \neg B_{1,1}\} .$$

Which query is entailed?

- A. $P_{1,2}$
- B. $P_{2,1}$
- C. $\neg P_{1,2} \wedge \neg P_{2,1}$
- D. $P_{1,2} \vee P_{2,1}$

Answer: No adjacent pit

Correct answer: C

$$\neg B_{1,1} \quad \text{and} \quad B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$$

imply

$$\neg(P_{1,2} \vee P_{2,1}) \equiv \neg P_{1,2} \wedge \neg P_{2,1}.$$

This is the same model-set idea, now in our AI environment.

From Meaning To Inference

So far, the main question has been semantic:

Which possible worlds survive the knowledge base?

Syntax:

Symbols and formulas

$B_{1,1}, P_{1,2}, \neg B_{1,1}$

Semantics:

Models where the formulas are true.

Entailment is about all surviving models.

Next:

We need procedures that compute consequences without checking every model by hand.

Inference: Model Checking and Theorem Proving

Model checking

- Enumerate or search possible worlds.
- Check whether all KB worlds satisfy the query.
- Clear semantics.
- Often exponential.

Theorem proving

- Apply inference rules to formulas.
- Search for a proof.
- Avoids listing every model explicitly.
- Needs soundness and completeness guarantees.

Entailment versus proof

Semantic entailment

$$KB \models \alpha$$

Every model of KB is also a model of α

Syntactic derivability

$$KB \vdash \alpha$$

α can be derived from KB using a proof procedure.

Sound and complete proof procedures make these two ideas line up.

Is $\models$ stronger than $\vdash$?

Not by itself. They live in different layers.

$\text{KB} \models \alpha$: semantics

A statement about meaning.

Every possible world that satisfies KB also satisfies α .

$\text{KB} \vdash \alpha$: proof

A statement about a proof system.

Starting from KB, the proof rules can derive α .

We judge a proof system by how well $\vdash$ matches $\models$.

Soundness and completeness

Soundness

Everything the proof procedure derives is actually entailed.

$$KB \vdash \alpha \Rightarrow KB \models \alpha$$

Completeness

Everything that is entailed can be derived by the proof procedure.

$$KB \models \alpha \Rightarrow KB \vdash \alpha$$

Inference methods: What counts as evidence?

Different methods can answer the same entailment question:

$$KB \models \alpha$$

Model checking

Enumerate complete truth assignments.

Evidence: every model of KB also satisfies α .

Forward chaining

Apply Horn-clause rules to known facts.

Evidence: the query is derived.

Resolution

Derive new clauses from old clauses.

Evidence: $KB \wedge \neg\alpha \vdash \square$.

DPLL

Search through partial truth assignments.

Evidence: no satisfying assignment exists for $KB \wedge \neg\alpha$.

Modus ponens

We saw the simplest version in the card test. Modus ponens is the basic “if the rule applies, take the conclusion” proof rule.

Read the rule below as: if all premises $p_1, \dots, p_k$ are known, and those premises imply q , then infer q .

$$\frac{p_1, \dots, p_k, \quad (p_1 \wedge \dots \wedge p_k) \rightarrow q}{q}$$

Example

$$\frac{K, K \rightarrow O}{O}$$

Role

- Workhorse of rule-based reasoning.
- Sound for propositional logic.
- Complete only under restrictions, such as definite clauses.

Definite clauses: easy rule chaining

Clause families

- A Horn clause has at most one positive literal.
- A definite clause has exactly one positive literal.

A definite clause can be written as:

$$(p_1 \wedge \dots \wedge p_k) \rightarrow q$$

A single positive literal q is the conclusion to add.

Example:

$$\text{Wet} \wedge \text{Weekday} \rightarrow \text{Traffic}$$

Why this restriction helps: If all premises are known, forward chaining has a clear next step: add the one conclusion.

$$\frac{\text{Wet, Weekday, Wet} \wedge \text{Weekday}}{\text{Traffic}}$$

This is efficient rule-based reasoning, but it covers only a restricted part of propositional logic.

Forward chaining

Idea

- Start with known facts.
- Fire rules whose premises are all known.
- Add their conclusions.
- Stop when the query is found or no new facts appear.

Office robot

$$K \rightarrow O$$

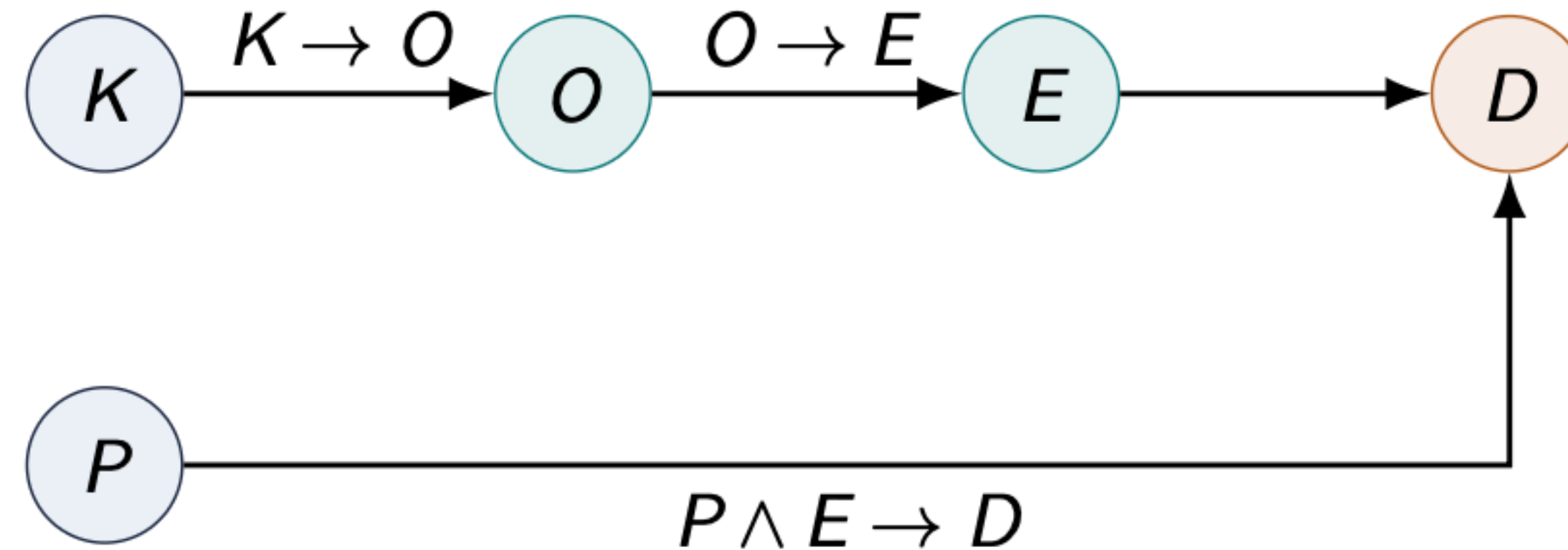
$$O \rightarrow E$$

$$P \wedge E \rightarrow D$$

$$K, P$$

Query: D.

Forward chaining trace



$$\{K, P\} \Rightarrow \{K, P, O\} \Rightarrow \{K, P, O, E\} \Rightarrow \{K, P, O, E, D\}$$

Since D is derived, forward chaining proves the query for this definite-clause KB.

Where rule chaining breaks

Definite clauses are good when each rule has one conclusion.

They are not good at rules with alternatives:

$$\text{Alarm} \rightarrow \text{Smoke} \vee \text{BreakIn}$$

As a clause

$$\neg \text{Alarm} \vee \text{Smoke} \vee \text{BreakIn}$$

Two positive literals: *Smoke* and *BreakIn*

Problem for chaining If Alarm is known, should we add *Smoke* or *BreakIn*?

Modus ponens does not choose between alternatives.

Why Horn clauses are limited

Forward chaining is sound for rules of this form:

$$p_1 \wedge \cdots \wedge p_k \rightarrow q$$

For definite-clause knowledge bases, it is also complete for atomic queries.

But this completeness result is for the restricted language, not for all of propositional logic.

General propositional logic allows clauses with alternatives, such as:

$$\neg \text{Alarm} \vee \text{Smoke} \vee \text{BreakIn}$$

This motivates general CNF-based methods. DPLL searches for a satisfying assignment; resolution searches for a contradiction proof.

Satisfiability, SAT, and SAT solvers

Definitions

- Valid: true in every model.
- Satisfiable: true in at least one model.
- Unsatisfiable: true in no model.

Examples

$$A \vee \neg A$$

valid

$$A \vee B$$

satisfiable

$$A \wedge \neg A$$

unsatisfiable

SAT is short for satisfiability and is usually pronounced “sat”. A **SAT solver** is a procedure that either finds a satisfying assignment or reports that no such assignment exists.

Entailment as unsatisfiability

$KB \models \alpha$ iff $KB \wedge \neg\alpha$ is unsatisfiable.

To prove KB entails α , show that KB together with $\neg\alpha$ has no model.

CNF-based methods can show this: DPLL by failing to find a model, resolution by deriving the empty clause.

This style of entailment test is called **refutation**: the negated query is refuted by contradiction.

If KB entails α , there is no KB -world where α is false.

Adding $\neg\alpha$ should make the clause set impossible to satisfy.

More symbols, and how to read them

Symbol	Say it as	Meaning
$KB \models \alpha$	“KB entails alpha” or “alpha is true in every model of KB”	every model of KB also makes α true
$KB \vdash \alpha$	“KB proves alpha” or “KB derives alpha”	a proof system can derive α from KB
$\square$	“empty clause”, “box”, or “contradiction”	a clause with no literals; it is false

A **refutation** is a proof by contradiction:
assume the query is false, then derive $\square$.

Literals and clauses

Literal

A symbol or its negation.

$$A, \neg A, O, \neg O$$

Clause

A disjunction of literals.

$$\neg K \vee O \vee E$$

CNF

A conjunction of clauses.

$$(\neg K \vee O) \wedge (\neg O \vee E) \wedge K$$

Conjunctive Normal Form (CNF)

$$(A \vee B \vee \neg C) \wedge (\neg B \vee D) \wedge (C)$$

CNF formula

- conjunction of clauses;
- each clause is a disjunction of literals;
- each literal is a symbol or negated symbol.

Clause set view

$$\{ \{A, B, \neg C\}, \{ \neg B, D \}, \{C\} \}$$

This is the representation used by DPLL and resolution.

CNF conversion rules

Step	Rewrite
Eliminate biconditional	$f \leftrightarrow g \equiv (f \rightarrow g) \wedge (g \rightarrow f)$
Eliminate implication	$f \rightarrow g \equiv \neg f \vee g$
Move negation inward	$\neg(f \wedge g) \equiv \neg f \vee \neg g$
Move negation inward	$\neg(f \vee g) \equiv \neg f \wedge \neg g$
Remove double negation	$\neg \neg f \equiv f$
Distribute	$f \vee (g \wedge h) \equiv (f \vee g) \wedge (f \vee h)$

These rewrites preserve models: the old formula and the new formula are equivalent.

CNF conversion example

Convert $(\text{Summer} \rightarrow \text{Snow}) \rightarrow \text{Bizarre}$

$$\begin{aligned}(\text{Summer} \rightarrow \text{Snow}) \rightarrow \text{Bizarre} &\equiv \neg(\neg\text{Summer} \vee \text{Snow}) \vee \text{Bizarre} \\ &\equiv (\text{Summer} \wedge \neg\text{Snow}) \vee \text{Bizarre} \\ &\equiv (\text{Summer} \vee \text{Bizarre}) \wedge (\neg\text{Snow} \vee \text{Bizarre})\end{aligned}$$

The final line is CNF: a conjunction of two clauses.

Question: Is this CNF?

Which formula is in CNF?

A. $(A \wedge B) \vee C$

B. $(A \vee \neg B) \wedge (C \vee D)$

C. $A \rightarrow B$

D. $\neg(A \vee B) \wedge C$

Answer: clause conjunction

Correct answer: **B**.

$$(A \vee \neg B) \wedge (C \vee D)$$

Each top-level part is a clause, and each clause is a disjunction of literals.

D is equivalent to CNF after rewriting, but it is not already in CNF because the negation is applied to a compound formula.

Two ways to use CNF

To test whether $KB \models \alpha$,
build:

$$S = \text{CNF}(KB \wedge \neg \alpha)$$

Now there are two different algorithmic views.

DPLL:

Try to construct a model of S (that is: find a satisfying assignment for S).

If all branches fail, S is unsatisfiable, so $KB \models \alpha$.

Resolution:

Derive new clauses from S .

If $\square$ is derived, then S is unsatisfiable, so $KB \models \alpha$.

What kind of method is DPLL?

DPLL stands for **Davis–Putnam–Logemann–Loveland**.

It is a complete SAT procedure: given a CNF formula, try to fill in truth values so every clause becomes true.

$$(A) \wedge (\neg A \vee B) \wedge (\neg B \vee C)$$

What it searches over

Partial truth assignments:

$$\{\}, \{A : T\}, \{A : T, B : T\}$$

What it returns

SAT: a satisfying assignment.

UNSAT: no assignment can satisfy the formula.

DPLL is a smarter truth-table search

A truth table checks every complete row: 2^n rows for n propositional symbols.

DPLL tries to build one row at a time, but often stops before the row is complete.

Situation	DPLL action
Clause is already true	keep searching under this branch
Clause is false	abandon this branch immediately
Clause has one possible way left	make that forced assignment
Nothing is forced	guess a symbol value and branch

It still has exponential worst case, but avoids many rows that cannot work.

What DPLL checks at each search node

At a search node, DPLL has a CNF formula and a partial assignment.

$$\{K : T, O : T\}$$

The table shows possible clause statuses under that assignment.

Clause	Status	Meaning for this branch
$\neg K \vee O$	satisfied	already true
$\neg O \vee E$	unit	E must be true
$\neg K$	contradicted	this branch fails
$E \vee P$	unresolved	no forced value yet

The key idea: a partial assignment can already force the next value or prove that this branch cannot become a model. If any clause is contradicted, DPLL stops that branch immediately.

The DPLL loop

Repeat these steps at each search node:

1. Simplify the clauses using the current partial assignment.
2. If every clause is satisfied, return SAT and the assignment.
3. If any clause is contradicted, this branch returns failure.
4. If a unit clause appears, assign its remaining literal to make it true.
5. If a pure literal appears, assign it so all clauses containing it become true.
6. Otherwise, choose an unassigned symbol and try true versus false.

If one guessed branch fails, DPLL backtracks and tries the other value. If all branches fail, the formula is UNSAT.

Unit propagation

A unit clause has exactly one unresolved literal.

$$(\neg K \vee O) \wedge K$$

If K is true, then $\neg K \vee O$ becomes:

$$F \vee O$$

so O must be true.

Unit propagation repeatedly applies forced assignments before branching.

DPLL trace: entailment chain

To test:

$$\{A \rightarrow B, B \rightarrow C, A\} \models C$$

check whether:

$$(\neg A \vee B) \wedge (\neg B \vee C) \wedge A \wedge \neg C$$

is satisfiable. DPLL returns UNSAT, so the entailment holds.

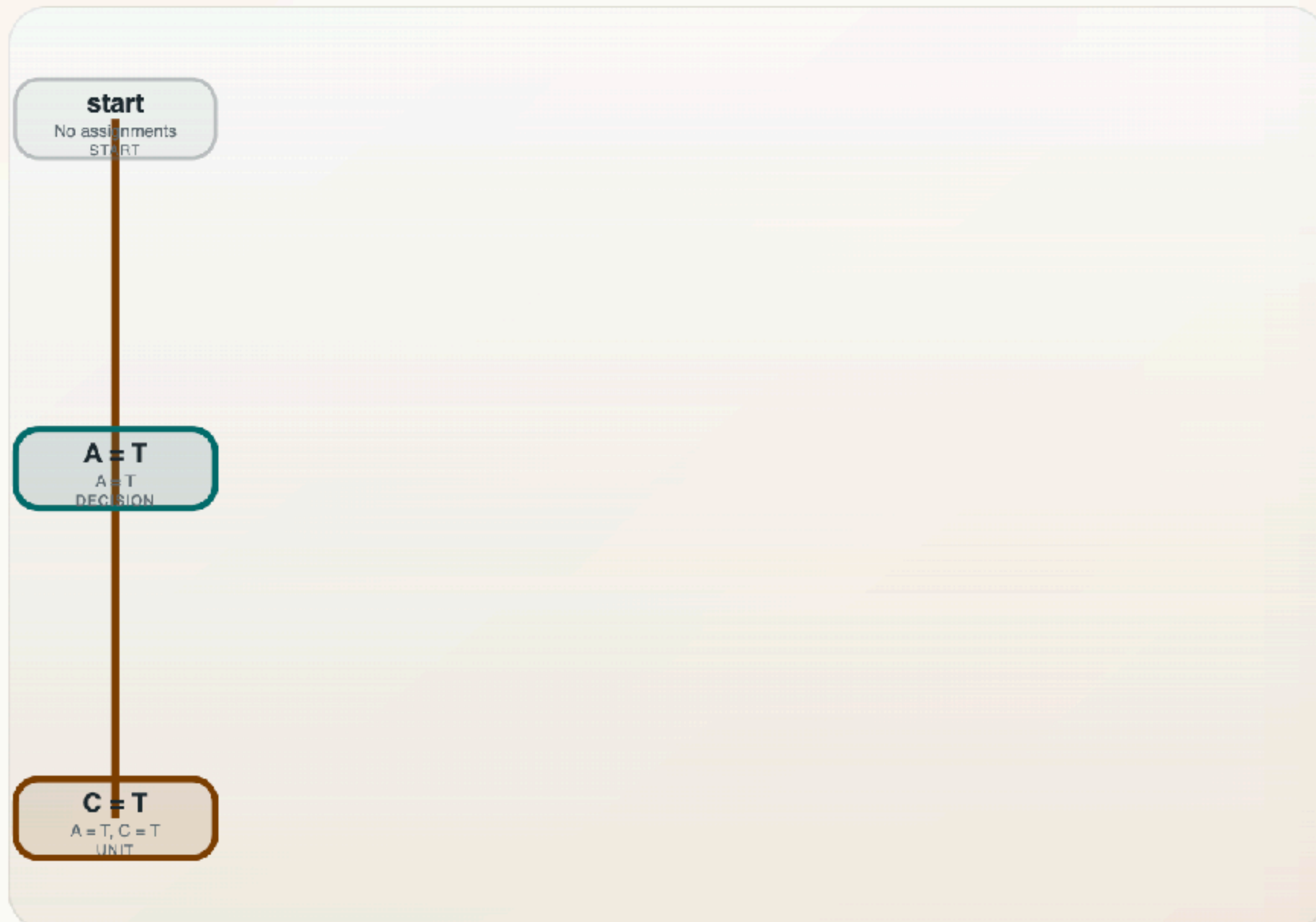
Step	Forced assignment	Reason
1	$A = T$	unit clause A
2	$B = T$	$\neg A \vee B$ becomes B
3	$C = T$	$\neg B \vee C$ becomes C
4	contradiction	$\neg C$ is also a unit clause

Logic Demo - Visual DPLL

A small satisfiable CNF where one decision quickly leads to a satisfying assignment.

Search Tree

Each node is a partial assignment, showing where DPLL branches and where literals become forced.



Clause State

The clause list shows which clauses are satisfied, unresolved, unit, or contradicted under the current assignment.



SATISFIED

3

UNRESOLVED

0

UNIT

0

CONTRADICTED ASSIGNED

0

2 / 3

Current assignment

A = T

decision

C = T

unit from C2

CNF clauses

C1 (A $\vee$ B)

SATISFIED

A

B

C2 ($\neg$ A $\vee$ C)

SATISFIED

 $\neg$ A

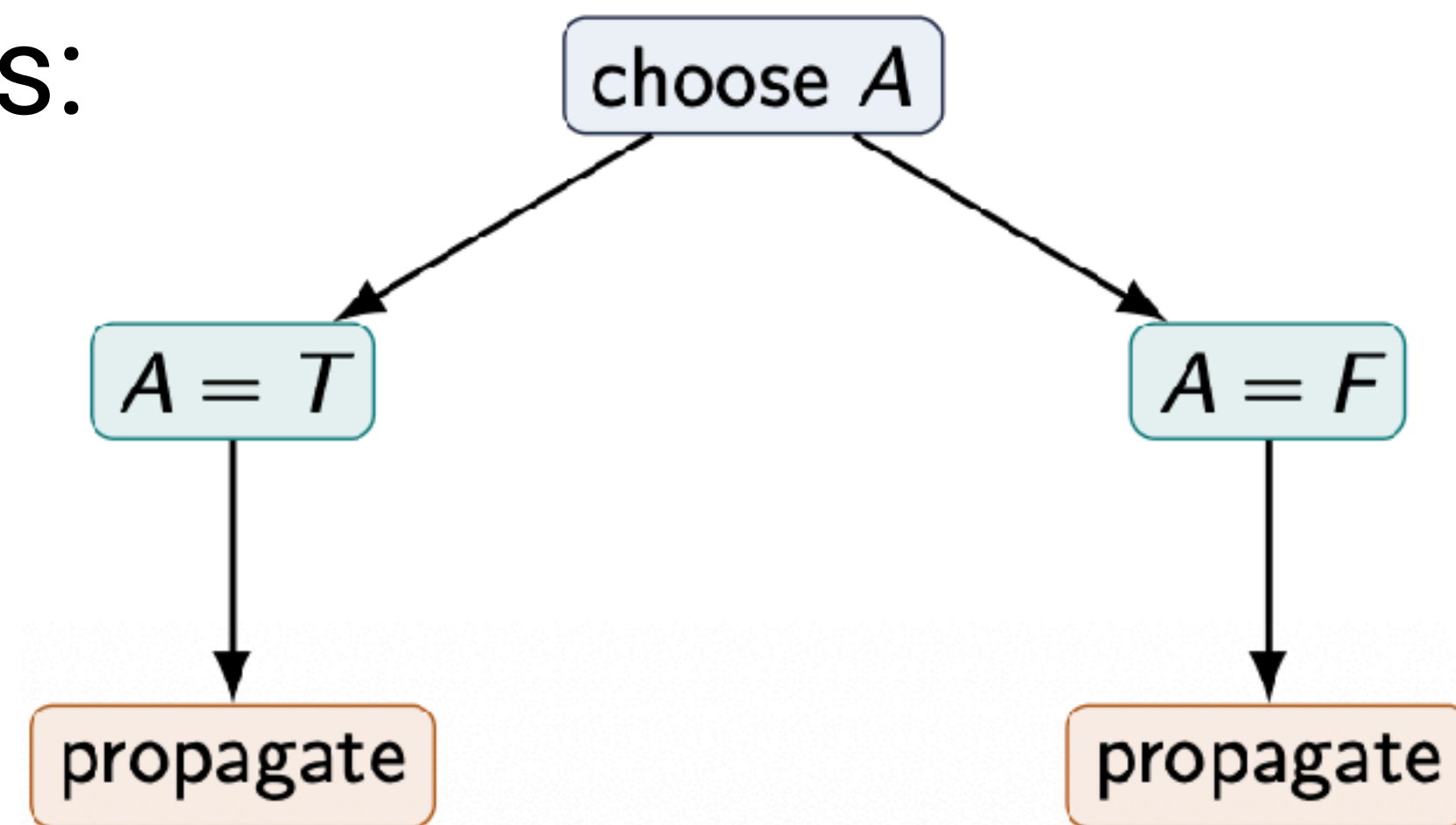
C

Branching when nothing is forced

Sometimes no unit clause or simple forced move exists.

$$(A \vee B) \wedge (\neg A \vee C) \wedge (\neg B \vee C)$$

DPLL then branches:



Branching is where the exponential worst case can appear.

DPLL versus resolution

DPLL: model search

- SAT decision procedure.
- State is a partial truth assignment.
- Move: assign, propagate, branch, backtrack.
- Output is a model, or UNSAT.

Resolution: proof search

- Refutation proof system.
- State is a clause set plus derived clauses.
- Move: resolve two clauses.
- Output is a proof trace ending in $\square$.

For entailment, both use $S = \text{CNF}(\text{KB} \wedge \neg\alpha)$. DPLL shows S has no model; resolution derives contradiction.

Question: DPLL

In DPLL, what should happen when a clause becomes unit?

- A. Assign its remaining literal so that the clause is satisfied.
- B. Delete the whole knowledge base.
- C. Immediately derive the empty clause.
- D. Ignore it until all other variables are assigned.

Answer: A

Correct answer: A.

$$\neg A \vee B$$

If A is already true, then the clause becomes:

$$B$$

DPLL must set B to true if this branch is to remain satisfiable.

Why resolution

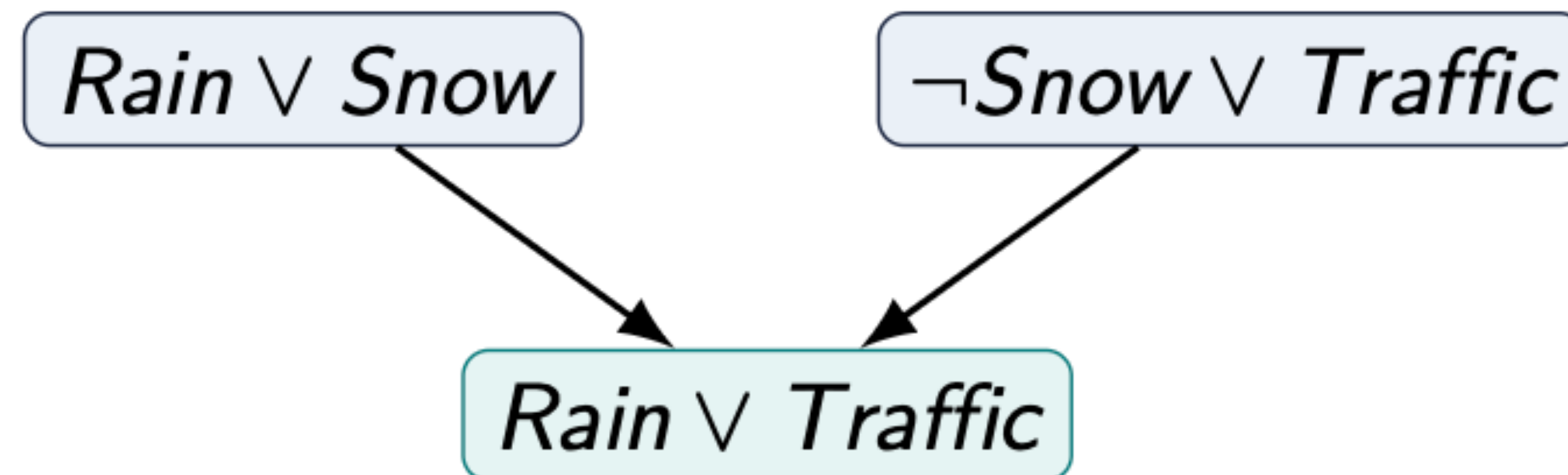
Formula class	Inference rule	Complete?
Full propositional logic	Modus ponens	No
Definite clauses only	Modus ponens	Yes, for atomic queries
Full propositional logic in CNF	Resolution	Yes, for refutation

Resolution is the proof method for arbitrary propositional clauses.

Resolution cancellation idea

$$\frac{\text{Rain} \vee \text{Snow}, \quad \neg\text{Snow} \vee \text{Traffic}}{\text{Rain} \vee \text{Traffic}}$$

The two parent clauses contain complementary literals: *Snow* and $\neg\text{Snow}$.



If both parent clauses are true, the resolvent must be true.

The resolution rule

$$\frac{f_1 \vee \dots \vee f_n \vee p, \quad \neg p \vee g_1 \vee \dots \vee g_m}{f_1 \vee \dots \vee f_n \vee g_1 \vee \dots \vee g_m}$$

- p and $\neg p$ are complementary literals.
- The conclusion is the resolvent.
- A resolution step does not replace the two parent clauses.
- It adds the resolvent to the clause set.
- The proof succeeds when the empty clause appears.

Empty clause and empty clause set

Empty clause

$\square$

A disjunction over zero literals.
It means contradiction: false.

Empty set of clauses

$\{\}$

A conjunction over zero clauses.
It means trivially true.

Resolution refutation algorithm

To prove KB entails α , show that KB together with $\neg\alpha$ has no model. Resolution shows this by deriving the empty clause.

1. Build $\text{KB} \wedge \neg\alpha$.
2. Convert the result to CNF.
3. Treat each CNF clause as a set of literals.
4. Repeatedly resolve pairs of clauses and add resolvents.
5. If $\square$ appears, return entailed.
6. If no new clauses appear, return not entailed.

Worked example

To prove KB entails C , show that KB together with $\neg C$ has no model. Resolution shows this by deriving the empty clause.

Knowledge base

$$A \rightarrow B$$

$$B \rightarrow C$$

$$A$$

Query: C

CNF for $\text{KB} \wedge \neg C$

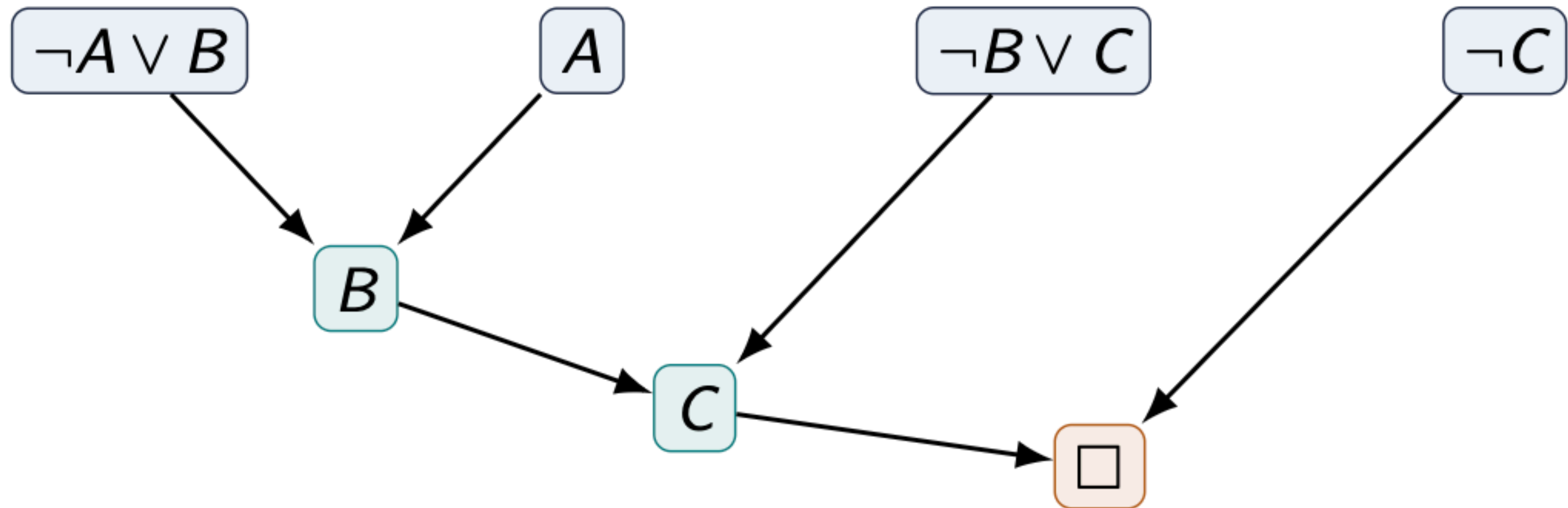
$$\{ \neg A \vee B, \neg B \vee C, A, \neg C \}$$

Refutation: proof table

Step	Clause	Reason
1	$\neg A \vee B$	from $A \rightarrow B$
2	$\neg B \vee C$	from $B \rightarrow C$
3	A	fact
4	$\neg C$	negated query
5	B	resolve 1 and 3
6	C	resolve 2 and 5
7	$\square$	resolve 4 and 6

$$KB \wedge \neg C \vdash \square \Rightarrow KB \models C$$

Refutation: proof diagram

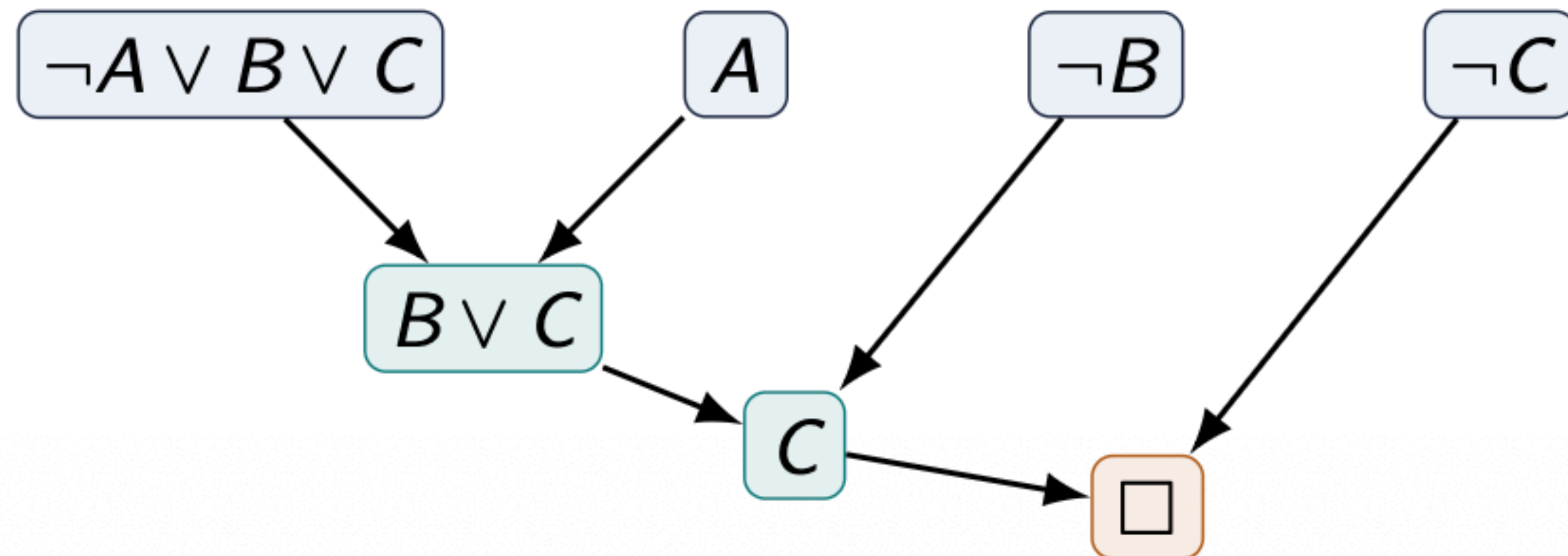


Worked refutation: disjunctive example

$$KB = \{A \rightarrow (B \vee C), A, \neg B, \neg C\}$$

Convert:

$$\{\neg A \vee B \vee C, A, \neg B, \neg C\}$$



Resolution on the Wumpus world

Revisiting:

$$B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$$

In CNF clauses:

$$\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$$

$$\neg P_{1,2} \vee B_{1,1}$$

$$\neg P_{2,1} \vee B_{1,1}$$

An observation $\neg B_{1,1}$ should let us prove $\neg P_{1,2}$ and $\neg P_{2,1}$.

Refutation query for Wumpus rule

We observe $\neg B_{1,1}$. Our **query** is about the pit, to prove $\text{KB} \models \neg P_{1,2}$.

There is no breeze in [1,1]. Does the KB force 'no pit' in [1,2]?

For refutation, add the negated query $P_{1,2}$ and try to derive $\square$.

$$1. \neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$$

$$2. \neg P_{1,2} \vee B_{1,1}$$

$$3. \neg P_{2,1} \vee B_{1,1}$$

$$4. \neg B_{1,1}$$

$$5. P_{1,2} \quad \text{negated query.}$$

The proof derives $B_{1,1}$, which contradicts $\neg B_{1,1}$.

Wumpus refutation: proof table

Step	Clause	Reason
1	$\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$	from the Wumpus rule
2	$\neg P_{1,2} \vee B_{1,1}$	from the Wumpus rule
3	$\neg P_{2,1} \vee B_{1,1}$	from the Wumpus rule
4	$\neg B_{1,1}$	observation
5	$P_{1,2}$	negated query
6	$B_{1,1}$	resolve 2 and 5
7	$\square$	resolve 4 and 6

$$KB \wedge P_{1,2} \vdash \square \quad \Rightarrow \quad KB \models \neg P_{1,2}.$$

Resolution strategy matters

Resolution is complete for refutation, but not every resolvent is useful.

$$B_{1,1} \vee P_{2,1} \vee \neg B_{1,1}$$

is always true because it contains $B_{1,1} \vee \neg B_{1,1}$.

In hand proofs, choose clauses that move towards contradiction:

- use the negated query;
- use observed facts;
- resolve away literals that block a direct contradiction

Resolution is sound

If both clauses are true:

$$(\text{Rain} \vee \text{Snow}) \wedge (\neg \text{Snow} \vee \text{Traffic})$$

then one of these cases must hold:

- *Snow* is true, so the second clause forces *Traffic*.
- *Snow* is false, so the first clause forces *Rain*.

Therefore:

$$\text{Rain} \vee \text{Traffic}$$

must be true. Resolution never derives a false consequence from true parent clauses.

Resolution is complete for refutation

Let S be any finite propositional clause set.

S is unsatisfiable iff $S \vdash_{\text{Res}} \square$.

- If a finite propositional clause set is unsatisfiable, resolution can derive the empty clause.
- This is refutation completeness.
- To prove $\text{KB} \models \alpha$, prove $\text{CNF}(\text{KB} \wedge \neg \alpha) \vdash_{\text{Res}} \square$.

What resolution costs

Good news

- Sound.
- Refutation-complete for propositional logic.
- One uniform rule.
- Works after CNF conversion.

Cost

- Naive resolution can generate many clauses.
- Worst-case proofs can be exponential.
- Practical systems need good strategies.

This is the price of moving beyond efficient definite-clause reasoning.

Question: One resolution step

Resolve

$$A \vee B \vee \neg C \qquad C \vee D$$

on C . What is the resolvent?

- A. $A \vee B \vee D$
- B. $A \vee B \vee \neg C \vee C \vee D$
- C. $A \vee B$
- D. D

Answer: Remove complementary literals

Correct answer: A.

$$\frac{A \vee B \vee \neg C \quad C \vee D}{A \vee B \vee D}$$

The complementary pair is $\neg C$ and C .

Keep the remaining literals from both clauses.

Question: Does the refutation prove entailment?

We want to test whether $KB \models Q$.

After adding $\neg Q$ and converting to CNF, resolution derives $\square$.

What should we conclude?

- A. $KB \models Q$
- B. $KB \models \neg Q$
- C. KB is satisfiable and says nothing about Q
- D. Resolution has failed

Answer: The query is entailed

Correct answer: A.

$$KB \wedge \neg Q \vdash \square$$

means there is no model where KB is true and Q is false.

Therefore every model of KB is a model of Q.

Where propositional logic runs out

Suppose the lab has many people, robots, rooms, and keycards.

Propositional version

HasKeycard_Robot1

CanEnter_Robot1_Lab

HasKeycard_Alice

CanEnter_Alice_Lab

HasKeycard_Bob

CanEnter_Bob_Lab

Repeated rule pattern

$\text{HasKeycard_R1} \rightarrow \text{CanEnter_R1_Lab}$

$\text{HasKeycard_Alice} \rightarrow \text{CanEnter_Alice_Lab}$

$\text{HasKeycard_Bob} \rightarrow \text{CanEnter_Bob_Lab}$

Propositional logic can represent each case, but it cannot state the general pattern directly.

First-order logic (FOL)

First-order logic adds structure inside sentences.

Vocabulary

Objects: Robot1, Alice, Lab, Parcel7

Predicates: HasKeycard(x),
CanEnter(x, r).

Variables: x, r, p .

Quantifiers: $\forall, \exists$.

General rule

$\forall x(\text{HasKeycard}(x) \rightarrow \text{CanEnter}(x, \text{Lab}))$

One sentence can describe a whole family of propositional cases.

Objects, predicates, and atomic sentences

A predicate applied to objects gives an atomic sentence.

FOL expression	Informal reading
<i>HasKeycard(Robot1)</i>	Robot1 has a keycard.
<i>CanEnter(Robot1, Lab)</i>	Robot1 can enter the lab.
<i>Delivered(Parcel7, Lab)</i>	Parcel7 has been delivered to the lab.
<i>Locked(Lab)</i>	The lab is locked.

Functions name objects from other objects, for example *Owner(Keycard7)*. We will mostly use constants and predicates.

Universal quantification

$$\forall x((\text{Robot}(x) \wedge \text{HasKeycard}(x)) \rightarrow \text{CanEnter}(x, \text{Lab}))$$

Reading:

For every object x , if x is a robot and x has a keycard, then x can enter the lab.

If the domain contains *Robot1*, *Alice*, and *Parcel7*, the rule applies to each object.

For objects that are not robots, the antecedent is false.

Existential quantification

$$\exists r(\text{Room}(r) \wedge \text{Locked}(r))$$

Reading:

There is at least one object r such that r is a room and r is locked.

Domain:

{Office, Lab, Corridor}

If $\text{Room}(\text{Lab})$ and $\text{Locked}(\text{Lab})$ are true, $\exists r(\text{Room}(r) \wedge \text{Locked}(r))$ is true.

The existential statement does not say which room unless the knowledge base contains more information.

Quantifier scope

Usual intended rule

$\forall x(\text{Robot}(x) \rightarrow \text{CanEnter}(x, \text{Lab}))$

Every robot can enter the lab.

It is clearer to always write parentheses around the scope of a quantifier.

Different scope

$(\forall x \text{ Robot}(x)) \rightarrow \text{CanEnter}(x, \text{Lab})$

If everything in the domain is a robot, then Robot1 can enter the lab.

Translating the office robot domain

Every robot with a keycard can open the lab door.

$$\forall x((\text{Robot}(x) \wedge \text{HasKeycard}(x)) \rightarrow \text{CanOpen}(x, \text{LabDoor}))$$

Anything that can open the lab door can enter the lab.

$$\forall x(\text{CanOpen}(x, \text{LabDoor}) \rightarrow \text{CanEnter}(x, \text{Lab}))$$

A parcel carried by something that enters the lab can be delivered to the lab.

$$\forall x \forall p((\text{Parcel}(p) \wedge \text{Carries}(x, p) \wedge \text{CanEnter}(x, \text{Lab})) \rightarrow \text{CanDeliver}(x, p, \text{Lab}))$$

First-order logic facts

Fact set

Robot(Robot1)

HasKeycard(Robot1)

Parcel(Parcel7)

Carries(Robot1, Parcel7)

FOL lets one rule apply to many possible robots and parcels.

It avoids writing a separate propositional rule for every object combination.

Here FOL is used to show how object-level knowledge can be written compactly. The proof method in this lecture remains propositional resolution.

FOL question

Which formula best says: every robot carrying a parcel can enter the lab?

- A. $\forall x(\text{Robot}(x) \rightarrow \text{CarryingParcel}(x) \wedge \text{CanEnter}(x, \text{Lab}))$
- B. $\forall x((\text{Robot}(x) \wedge \text{CarryingParcel}(x)) \rightarrow \text{CanEnter}(x, \text{Lab}))$
- C. $\exists x((\text{Robot}(x) \wedge \text{CarryingParcel}(x)) \rightarrow \text{CanEnter}(x, \text{Lab}))$
- D. $\text{Robot}(x) \wedge \text{CarryingParcel}(x) \rightarrow \text{CanEnter}(x, \text{Lab})$

FOL answer

Correct answer: **B**.

$$\forall x((\text{Robot}(x) \wedge \text{CarryingParcel}(x)) \rightarrow \text{CanEnter}(x, \text{Lab}))$$

- A says every robot is carrying a parcel and can enter the lab.
- C only says there is at least one object for which the implication is true.
- D uses x without quantifying it.

Is logic AI?

Logic is one important tradition within AI, not the whole field.

Why it is AI

- It represents knowledge explicitly.
- It supports reasoning from facts to consequences.
- It can plan, diagnose, verify, and prove.
- It gives inspectable explanations.

Why it is not all of AI

- Perception is often noisy and uncertain.
- Many domains are too large to encode by hand.
- Learning from data uses different tools.
- Modern systems often combine logic, search, and statistics.

Can a proof system discover new knowledge?

Yes, if the result follows from the axioms and the proof can be checked.

Automated theorem proving searches through possible proof steps, often far beyond what a person would try manually.

Example: Robbins conjecture

- Open since the 1930s
- Asked whether every Robbins algebra is Boolean.
- Proved in 1996 by EQP, an automated theorem prover developed at Argonne.

What was discovered?

- The system found a proof no human had previously found.
- Humans supplied the axioms, representation, and search setup.
- The proof object could then be checked and studied.

What to remember

1. Keep the three layers separate: syntax, semantics, and proof/search.
2. A knowledge base is a set of sentences about possible worlds.
3. Propositional logic uses true/false symbols and logical connectives.
4. Entailment means every model of the KB also satisfies the query.
5. Proof procedures manipulate formulas rather than enumerate all models.
6. Definite-clause reasoning is efficient but restricted.
7. CNF represents knowledge as a set of clauses.
8. Resolution proves entailment by refuting KB together with the negated query.
9. FOL adds objects, predicates, variables, and quantifiers so rules can describe many objects compactly.
10. Logic-based AI can discover consequences, including proofs humans have not yet found.